

Tutorial Sheet 5

Synchronization

AUTHOR

Operating Systems Teaching and Research Unit

PUBLISHED

21.06.2024

Task 1: Semaphore

Using semaphore, implement the following synchronization mechanisms.

1. Mutex

Define a structure `struct Mutex` and implement the following functions:

```
1 // initialize your struct Mutex
2 void mutex_init(struct Mutex *mutex);
3
4 void mutex_lock(struct Mutex *mutex);
5
6 void mutex_unlock(struct Mutex *mutex);
```

1. Barrier

Define a structure `struct Barrier` and implement the following functions:

```
1 // initialize your struct Barrier
2 void barrier_init(struct Barrier *barrier, int n_threads);
3
4 // wait until all the threads are waiting, then release the threads
5 void barrier_wait(struct Barrier *barrier);
```

Task 2: Mensa Ahornstraße

a)

In this task, we simulate the functionality of the Mensa Ahornstraße. We assume there are only two dishes, "Klassiker" and "Vegetarian". At the counter, both dishes are available. Only one person can be at the counter at a time, stating their food request. If the dish is available, a portion is handed over, the person pays, and then leaves the counter. Otherwise, the person waits at the counter until the dish is available. Whenever the kitchen prepares new portions, they are made available by the cafeteria staff.

Here is the current solution. The `arrivalStudent` function is started by different processes, `fillklassiker` and `fillVegetarian` are called by only one process.

```

1  #define KLASIKER 0
2  #define VEGETARIAN 1
3
4  int studentsAtCounter = 0;
5  int klassiker = 0;
6  int vegetarian = 0; // shared variables
7
8  void fillKlassiker(int n) {
9      klassiker += n;
10 }
11
12 void fillVegetarian(int n) {
13     vegetarian += n;
14 }
15
16 void arrivalStudent(int wish) {
17     while (studentsAtCounter > 0);
18     studentsAtCounter++;
19     if (wish == KLASIKER) {
20         while (klassiker == 0) {
21             // wait
22         }
23         klassiker--;
24     }
25     else if (wish == VEGETARIAN) {
26         while (vegetarian == 0) {
27             // wait
28         }
29         vegetarian--;
30     }
31     studentsAtCounter--;
32 }

```

Explain the term busy waiting in the context of this task and give two examples from the pseudocode.

b)

Synchronize the solution using mutex. Justify the placement of the critical sections.

c)

Instead of a mutex, synchronize the solution using semaphore. Justify the placement of the critical sections. You may need additional variables for correct synchronization.

Task 3: Deadlock

Given three resources: hard drive, card reader, and printer. Each process always needs exactly two of these resources exclusively. The following sequences for processes have been defined. All semaphores (hardDrive, cardReader, printer) are mutexes, initialized to 1. Does this solution work? If you find an error or problem, describe how to fix it.

Process A:

```
1 while (condition) {
2     sem_wait(hardDrive);
3     sem_wait(cardReader);
4     useDevices();
5     sem_post(hardDrive);
6     sem_post(cardReader);
7 }
```

Process B:

```
1 while (condition) {
2     sem_wait(printer);
3     sem_wait(hardDrive);
4     useDevices();
5     sem_post(printer);
6     sem_post(hardDrive);
7 }
```

Process C:

```
1 while (condition) {
2     sem_wait(cardReader);
3     sem_wait(printer);
4     useDevices();
5     sem_post(cardReader);
6     sem_post(printer);
7 }
```

Task 4: Sleeping Barber

In this problem, we have a single barber working in a barbershop. The barbershop has a waiting room that can hold up to 20 customers. Only one customer at a time can be in the barber's chair for a haircut. If there are no customers in the shop, the barber, who often gets tired from working too much, will go to sleep. However, if a customer enters the barbershop and finds the barber sleeping, the customer will wake up the barber.

You need to implement the routines for both the barber and the customers, using proper synchronization mechanisms to ensure the requirements are met.

Requirements:

1. The barber should go to sleep if there are no customers waiting.
2. A customer should wake up the barber if they find him sleeping.
3. Only one customer should be in the barber's chair at a time.
4. The waiting room should not hold more than 20 customers at a time.

Questions:

1. Identify the constants that need to be declared.
2. Determine the number of semaphores required and their initial values.
3. Specify the number of mutexes required.

Your implementation should ensure proper synchronization between the barber and the customers, adhering to the constraints mentioned above.